



UNIVERSITY OF
LEICESTER

**School of Computing and
Mathematical Sciences**

CO7201 Individual Project

Preliminary Report

**Beyond Static Retrieval: Implementing an
Autonomous Multi-Agent Framework for
Complex Domain-Specific Reasoning**

Aamir Nasir

aan23@student.le.ac.uk

249045457

Project Supervisor: Prof. Huiyu Zhou

Principal Marker: Dr. Robert Free

Word Count: 1741

27th February 2026

DECLARATION

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name - Aamir Nasir

Date – 27th February 2026

Contents

1. Aims and Objectives	3
1.1 Motivation.....	3
1.2 Main Challenges	3
2. Requirements.....	3
2.1 Essential Requirements.....	3
2.2 Recommended Requirements	3
2.3 Optional Requirements	4
3. Technical Specification.....	4
3.1 Core Logic and Frameworks.....	4
3.2 Models.....	5
3.3 Data Storage.....	5
4. Requirements Evaluation Plan	5
4.1 Automated Metrics.....	5
4.2 Pipeline Testing	5
4.3 Human Evaluation	5
5. Background Research and Reading list.....	6
5.1 Background Research	6
5.2 Reading List.....	6
6. Time-plan and Risk Plan.....	7
6.1 Time-plan.....	7
6.2 Risk Plan	7
7. References.....	8

1. Aims and Objectives

1.1 Motivation

Standard Retrieval-Augmented Generation (RAG) pipelines are highly effective for general queries, but they have a major blind spot when applied to complex fields like medicine. A standard RAG setup blindly trusts whatever the vector database returns. If the search step retrieves a paper that is only tangentially related to the user's prompt, the LLM doesn't know any better. It simply takes that flawed context and generates an answer, which directly leads to factual hallucinations.

This project attempts to solve that specific problem by moving from a static pipeline to an "Agentic" framework. The goal is to build a system that can grade its own homework. By introducing a self-correcting feedback loop, a dedicated evaluator agent can look at the retrieved documents and decide if they actually answer the question. If the documents are irrelevant, the system should autonomously reject them, rewrite the search query to be more specific, and try again.

1.2 Main Challenges

The core difficulty will be managing the state of the agents so the system doesn't get stuck in a loop. If the evaluator keeps rejecting the documents, the system could just query the database forever.

Additionally, biomedical data is full of complex multi-hop relationships. Simple cosine similarity searches often miss these links. Figuring out how to integrate graph-based retrieval to map these relationships, without making the system too slow to use, is going to be the main technical hurdle.

2. Requirements

2.1 Essential Requirements

- Build a baseline linear RAG pipeline using ChromaDB to act as a control.
- Implement a cyclic state machine with specific nodes for retrieving documents, grading them, rewriting queries, and generating the final response.
- Write the conditional routing logic so the system actually falls back and retries when the grader agent rejects a document.
- Process and index a test batch of PubMed abstracts.

2.2 Recommended Requirements

- Integrate a Neo4j graph database alongside the vector store to see if GraphRAG improves the retrieval of complex medical relationships.

- Use the RAGAS framework to automatically score the final outputs for faithfulness and relevance.
- Create a Streamlit web interface to visualize the reasoning trace of the agents.

2.3 Optional Requirements

- Run the intermediate grading and rewriting steps locally using Ollama and an open-source model like Llama-3 to cut down on API costs.
- Test the agent routing logic on a non-medical dataset to see if it generalizes.

3. Technical Specification

The architecture will be developed using a Python-based stack, prioritizing libraries built specifically for agentic control flow and semantic indexing.

3.1 Core Logic and Frameworks

- The entire backend will be written in Python. For the orchestration, standard LangChain chains are too linear. I am going to use LangGraph because it lets you define the workflow as a state machine. You define the agents as nodes and write explicit python functions for the edges to handle the cyclic routing.

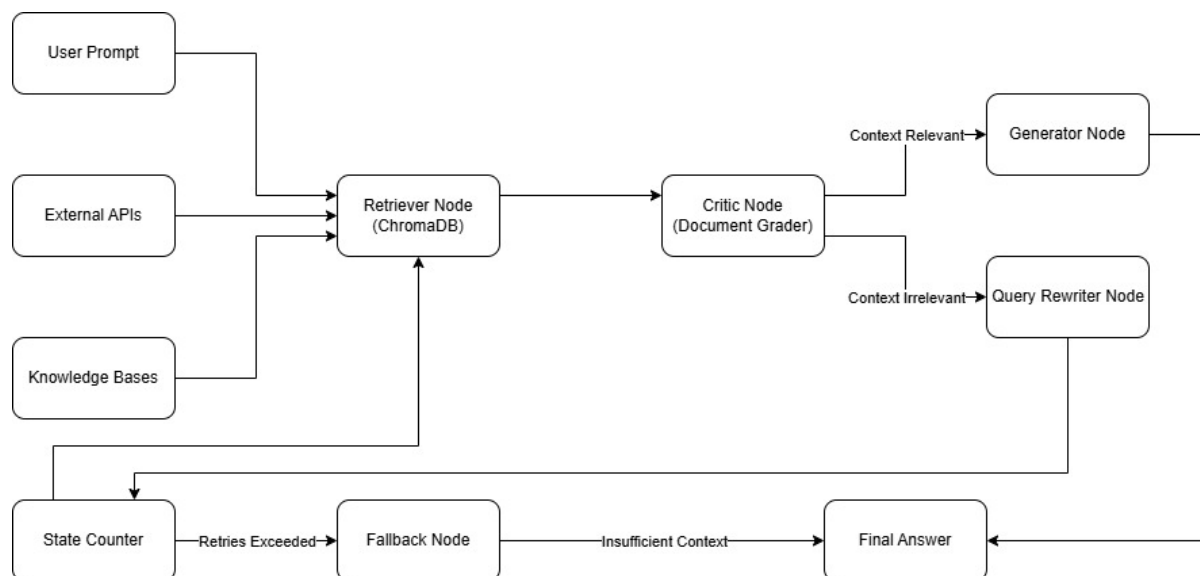


Figure 1: The proposed LangGraph state machine, showing the cyclic routing loop and the fallback mechanism.

3.2 Models

- I will use the OpenAI API (probably GPT-4o-mini) for the initial development just to ensure the routing logic works. For the embeddings, I'm planning to use the HuggingFace **all-MiniLM-L6-v2** sentence-transformer. It's lightweight enough to run locally without bottlenecking the retrieval step.

3.3 Data Storage

- ChromaDB will be used for the vector store. It is easy to set up locally and handles standard similarity searches well.
- For the GraphRAG component, I'm looking at Neo4j Community Edition. I haven't finalized the exact pipeline for extracting the entity relationships from the raw text into Cypher queries yet, but that is the target architecture.

4. Requirements Evaluation Plan

Evaluation will be conducted across three distinct phases, assessing both software integrity and the system's success in mitigating hallucinations.

4.1 Automated Metrics

To prove the system actually reduces hallucinations, I need hard numbers. I will run a set of about 50 test queries through the baseline RAG and the Agentic RAG. I plan to use RAGAS to score the outputs. I am specifically interested in the "Faithfulness" score (did the model hallucinate?) and the "Context Precision" score (did the grading agent successfully filter out the junk data?).

4.2 Pipeline Testing

I need to unit test the individual agent nodes to make sure they return the correct data structures. The Query Rewriter, for example, needs to return a clean string, not a conversational response. I'll also stress-test the cyclic graph by hardcoding a limit of 3 retries to ensure it doesn't infinite loop.

4.3 Human Evaluation

RAGAS is good for bulk testing, but I also want my supervisor to manually review about 20 of the more complex medical outputs. This will be a sanity check to ensure the system's reasoning actually makes sense clinically.

5. Background Research and Reading list

5.1 Background Research

When looking into how to fix the hallucination issue, it became pretty clear that standard RAG just isn't enough for medical texts. The original RAG paper by Lewis et al. [3] proved that fetching external data helps, but it assumes the vector database always returns the right documents. In reality, especially with medical terms, cosine similarity often pulls up documents that use the same words but mean completely different things.

To get around this, a lot of recent papers are exploring "Agentic" systems that can double-check their own work. For example, the Self-RAG paper [1] talks about training models to output reflection tokens so they can grade their own context. Another approach is Corrective RAG (CRAG) [2], which uses a separate evaluator agent to score the documents and actually trigger web searches or rewrites if the context is bad. This is basically the routing logic I want to implement.

I've also been reading up on GraphRAG [5]. Flat vector databases are really bad at multi-hop reasoning (like tracking how a disease relates to a gene, and then to a drug). GraphSearch frameworks [7] show that having agents navigate a Knowledge Graph step-by-step works much better. So, the theoretical background for my project is essentially mashing these two concepts together: building a CRAG-style feedback loop that uses graph retrieval instead of just vector search.

5.2 Reading List

Prof. Zhou recommended that I need to draw my technical research strictly from top-tier academic venues rather than general tutorials and to use these specific sources to find the most up-to-date implementations of Agentic RAG and Knowledge Graphs:

1. ICLR (International Conference on Learning Representations)
2. NeurIPS (Neural Information Processing Systems)
3. ICML (International Conference on Machine Learning)
4. ICCV (International Conference on Computer Vision)
5. ECCV (European Conference on Computer Vision)
6. IEEE TPAMI (Transactions on Pattern Analysis and Machine Intelligence)
7. IJCV (International Journal of Computer Vision)
8. IEEE Transactions on Image Processing
9. JMLR (Journal of Machine Learning Research)
10. Machine Learning Journal (Springer)

6. Time-plan and Risk Plan

6.1 Time-plan

- **Weeks 1-2:** Project setup, literature review, and GitLab initialization.
- **Weeks 3-4:** Data processing. Getting the PubMed abstracts cleaned, chunked, and embedded into ChromaDB. Build the baseline RAG control script.
- **Weeks 5-6:** Build the LangGraph nodes. The main focus here will be prompt engineering the Document Grader agent so it scores relevance accurately.
- **Weeks 7-9:** Connect the nodes and write the routing logic. Get the working prototype ready for the Principal Marker Interview.
- **Weeks 10-11:** Integrate RAGAS, run the main evaluations, and start writing the report.
- **Weeks 12-13:** Build the Streamlit UI, finish the report, and submit the final code.
- **Week 14:** Viva prep.

6.2 Risk Plan

- **Infinite Loops:** If the grading agent's prompt is too strict, it will reject everything. I will mitigate this by adding a state counter to the graph that forces a fallback message ("Insufficient context") after 3 failed attempts.
- **Graph Database Issues:** Setting up Neo4j and extracting accurate Node-Edge triplets might take longer than expected. If it delays the core project, I will drop the GraphRAG requirement and rely entirely on ChromaDB. The multi-agent routing will still work.
- **API Costs:** Running multiple agents per query gets expensive fast. If costs spike, I will switch the intermediate reasoning steps to run locally via Ollama.
- **Data Loss:** Hardware failure or accidentally deleting my local ChromaDB instance could wipe out hours of embedding work. I will mitigate this by committing my code to the university GitLab repo at least three times a week and keeping a zipped backup of the raw PubMed dataset on my university OneDrive.

7. References

1. Akari Asai et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR 2024.
2. Yan, S., et al. (2024). Corrective Retrieval Augmented Generation. arXiv preprint.
3. Lewis, Patrick, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS, 2020.
4. Dawson, C. W. (2009). Projects in Computing and Information Systems: A student's guide. Pearson.
5. Edge, D. "From Local to Global: A Graph RAG Approach to Query-Focused Summarization." arxiv:2404.16130.
6. Es, S. RAGAS: Automated Evaluation of Retrieval Augmented Generation. 2023.
7. Peng, C., et al. (2025). GraphSearch: An Agentic Deep Searching Workflow for Graph Retrieval-Augmented Generation.
8. LangGraph Documentation. <https://langchain-ai.github.io/langgraph/>.